Dynamic Capacity Service for Improving CXL Pooled Memory Efficiency

Minho Ha, Junhee Ryu, Jungmin Choi, Kwangjin Ko, Sunwoong Kim, Sungwoo Hyun, Donguk Moon, Byungil Koh, Hokyoon Lee, Myoungseo Kim, Hoshik Kim, and Kyoung Park, *SK hynix Inc., Icheon, 17336, South Korea*

Compute Express Link (CXL) pooled memory is gaining attention from the industry as a viable memory disaggregation solution offering memory expansion and alleviating memory overprovisioning. One essential feature for the efficient use of the pooled memory is to dynamically allocate or release memory from the pool based on hosts' demands. We refer to this feature dynamic capacity service (DCS). This article introduces one of the industry's first DCS implementation for CXL pooled memory. We demonstrate fully functional DCS by implementing a field-programmable gate array-based CXL pooled memory prototype and full software stacks. Our experiment shows that DCS can substantially improve system memory utilization by dynamically allocating and releasing memory resources on demand. We also present the lessons learned from the DCS implementation.

With the emergence of data-intensive applications, the amount of data processed in computing systems is increasing exponentially.[1] A growing amount of data drives the demand for large-capacity and high-bandwidth memory. However, there are several challenges with the conventional systems.

Challenge 1: Memory Wall

The latest trends in computing systems indicate that improvement in memory performance has not kept pace with compute performance.[2,3] For memory-intensive workloads, a growing gap between compute and memory performance may hurt overall system performance.

A viable solution to expand memory bandwidth and capacity is to increase the number of memory channels in the central processing unit (CPU).[4] However, this requires CPU design overheads leading to problems of increased costs and reduced reliability. In addition, improving the data rate of dynamic random-access memory (DRAM) for higher memory bandwidth is challenging due to increased implementation complexity.

0272-1732 © 2023 IEEE
Digital Object Identifier 10.1109/MM.2023.3237756
Date of publication 17 January 2023; date of current version 13 March 2023.

Challenge 2: Memory Overprovisioning

As resource requirements (e.g., the number of CPU cores and memory size) of workloads changes over time and varies between hosts, a substantial amount of memory in cloud-scale systems is left stranded. For example, up to 25% of memory is stranded in Microsoft's Azure.[5] As a cost-effective way to alleviate such memory overprovisioning issues, memory disaggregation is grabbing attention from the industry, and various research enabling the technology is underway.

APPROACHES TO USING MEMORY EFFICIENTLY

Solution 1: Compute Express Link (CXL) Memory Expander

CXL, an industry-supported cache-coherent interconnect offering high-performance connectivity among various heterogeneous devices, allows device-attached memory to be used like host-attached memory.[6] Taking this feature, CXL memory expanders have been recently announced by major memory companies. With CXL memory expander, users can easily add memory through CXL ports, and enjoy the effect of expanding memory capacity and bandwidth.[4]

However, there is a limit to the number of CXL lanes that a single host processor can support. For example, the first CXL-enabled CPU can connect only up to four CXL devices. It means that, beyond certain

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

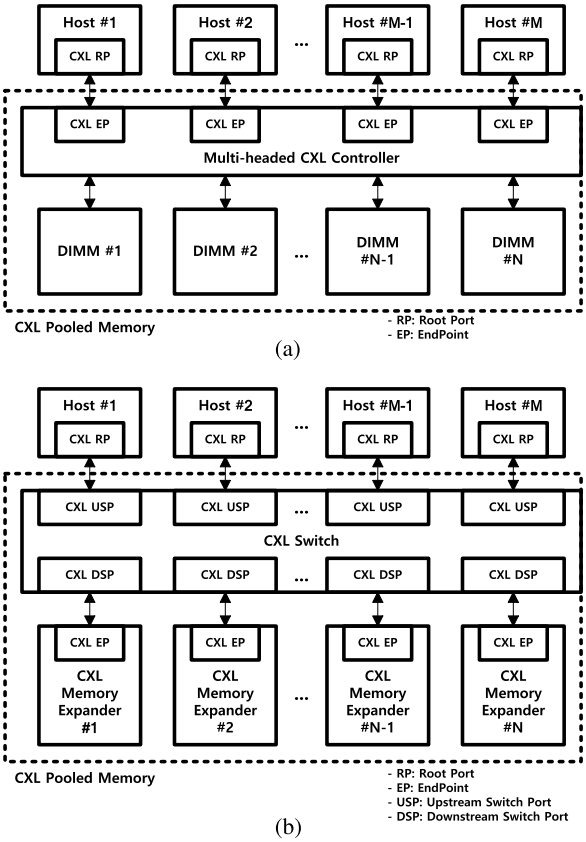

FIGURE 1. CXL pooled memory architecture.[4] (a) Multiheaded CXL device-based pooled memory. (b) CXL switch-based pooled memory.

level, memory expansion using CXL memory expander also needs scale out, which means user should add more server. Furthermore, since the CXL memory expander is directly connected to the single host processor, it is not free from the memory overprovisioning.

Solution 2: Memory Disaggregation

Memory disaggregation[5,7,8,9] is a method of physically separating memory node from compute node. Advantages of memory disaggregation are high memory utilization, transparent scalability, and efficient resource management. It is well known as a solution to alleviate memory overprovisioning. To maximize the benefits of memory disaggregation, support for coherent memory load/store semantics and hardware/software (HW/SW) co-design are required.

NEW APPROACH: CXL POOLED MEMORY

CXL pooled memory is a disaggregated memory system in which multiple hosts are connected through CXL to share a large memory pool.[4] CXL pooled memory combines the advantages of memory disaggregation and CXL memory expander, addressing the memory overprovisioning issue while offering memory expansion.

Architecture Topology

CXL pooled memory can be implemented by 1) connecting multiple hosts directly to multiheaded CXL memory devices or 2) incorporating a switch between hosts and CXL devices.[4] Figure 1(a) is an example of multiheaded CXL memory device-based pooled memory architecture where M hosts and N dual inline memory modules (DIMMs) are connected. Multiple hosts can be connected as the multiheaded CXL controller allows. Figure 1(b) is an example of switch-based pooled memory architecture where M hosts and N CXL memory expanders are connected through a CXL switch. The scalability of the pooled memory in Figure 1(b) will be further improved once CXL 3.0,[6] which support the connection of multiple CXL switches, is deployed. CXL pooled memory allows multiple configurations between hosts and memory devices. For example, multiple hosts can share resources within one DIMM (or CXL memory expander), or one host can use multiple DIMMs (or CXL memory expanders).

Benefits

CXL pooled memory has the following advantages by combining the features of CXL and memory disaggregation, such as memory load/store semantics, high scalability, and high resource utilization.[4,10]

Memory utilization improvement: CXL pooled memory enables not only memory expansion but also memory resource sharing between multiple hosts. Memory resource sharing allows one host to use memory resources that are not used by other hosts, which improves memory utilization and memory overprovisioning. Improving memory utilization is directly related to a total cost of ownership reduction. In addition, since memory resources are efficiently used, storage access is reduced, thereby reducing performance degradation due to swap.

Memory semantics: Because CXL supports accessing device-attached memory with load and store instructions, CXL pooled memory can be used in memory semantic way.[10] For the conventional disaggregated memory, memory access performance is greatly degraded due to the presence of overhead including unnecessary data movement for host CPU to access the remote memory.[5,7] However, since memory devices in CXL pooled memory can be accessed by load and store instructions just like the existing host-attached memory, a lot of overhead is eliminated.

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

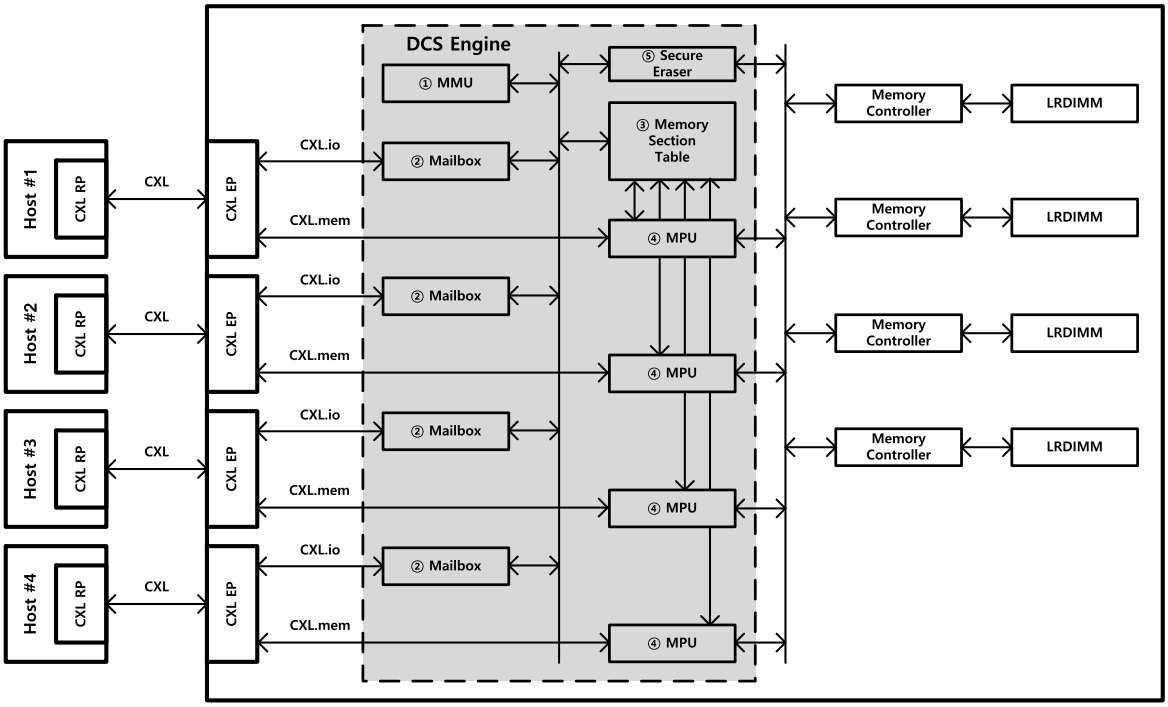

FIGURE 2. Multiheaded CXL memory hardware architecture with DCS engine.

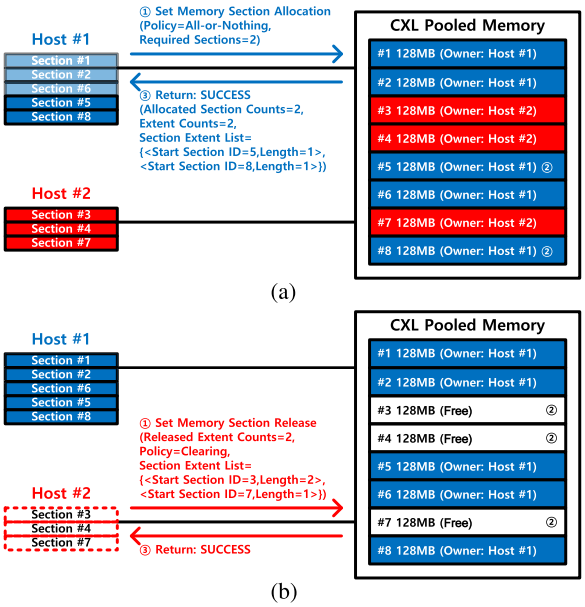

FIGURE 3. Examples of dynamic memory allocation and release by MMU. (a) Dynamic memory allocation. (b) Dynamic memory release.

High scalability: With CXL pooled memory, it is possible to expand the memory up to several TB through CXL switch beyond the limited number of CXL lanes for each host. If the concept of fabric through multiple switches supported by CXL 3.0[6] is applied, the scalability of CXL pooled memory will be increased further.

TABLE 1. DCS Mailbox command sets.

Opcode			
Command set bits [15:8]		**Command bits [7:0]**	
C0h	Dynamic capacity service	00 h	Get memory section status
		01 h	Set memory section allocation
		02 h	Set memory section release
		03 h	Set memory section size
		99 h	Clear section

CXL Pooled Memory Initiative

Thanks to the abovementioned advantages, the market of CXL pooled memory is expected to grow rapidly in the future. However, since the full potential of the CXL pooled memory cannot be fully exploited just by allowing multiple hosts to access the shared memory pool, such as architectures shown in Figure 1, we explore how the dynamically sharing the pooled memory capacity can maximize memory capacity utilization.

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

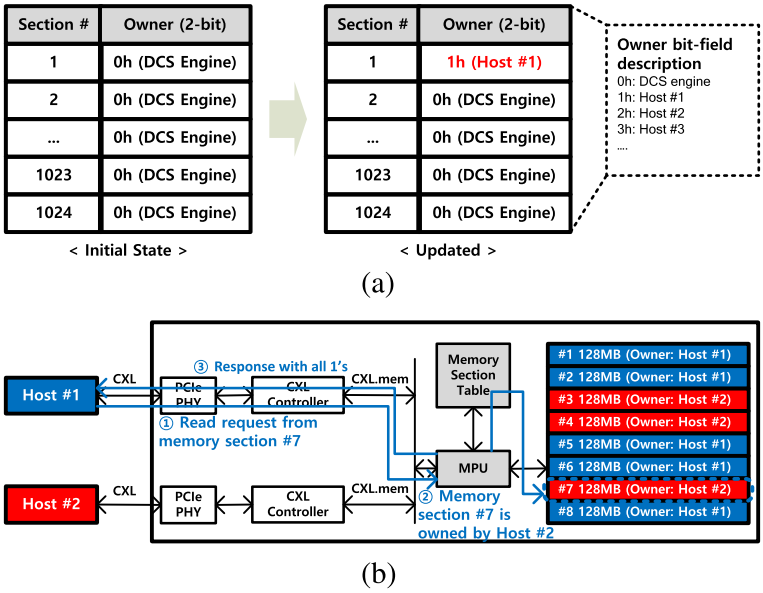

FIGURE 4. Operation example of DCS hardware components. (a) Example of memory section table update. (b) Example of transaction drop by MPU.

For example, if the CXL pooled memory regions are statically allocated to each host, the given memory resource may not be effectively used. To address this issue, the article introduces the dynamic capacity service, which is a feature to dynamically resize system memory by allocating and releasing CXL pooled memory correspond to the memory requirement of a host.

In addition, our research will cover the block data management, which can move or copy memory pages within the CXL pooled memory device. Also, performance monitor and reliability, availability, and serviceability functions may be additionally required, and we plan to focus on improving system performance with CXL pooled memory architecture through continuous research to discover the solutions.

DYNAMIC CAPACITY SERVICE

Conventionally, each host's memory capacity is statically assigned during system boot time. As individual hosts' working set size (WSS) changes over time, the entire system may experience memory underutilization or performance degradation caused by swap operations.

We present dynamic capacity service (DCS) addressing the abovementioned inefficiency by dynamically allocating and releasing CXL memory resources according to each host's varying WSS. We expect DCS will improve the memory utilization and performance of the multihost system with CXL pooled memory by avoiding unnecessary memory resource allocation.

We have designed DCS, an HW/SW-integrated CXL pooled memory solution based on the dynamic capacity device described in CXL 3.0,[6] and demonstrated

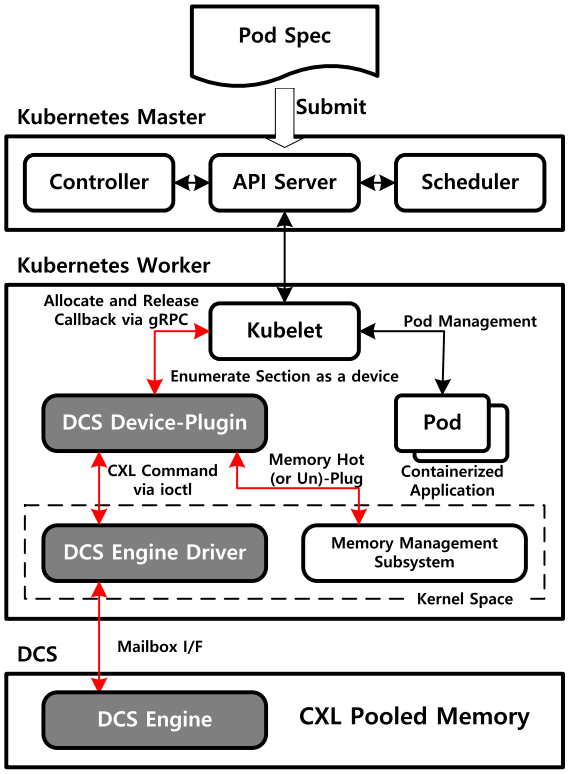

FIGURE 5. Software framework for dynamic capacity service.

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

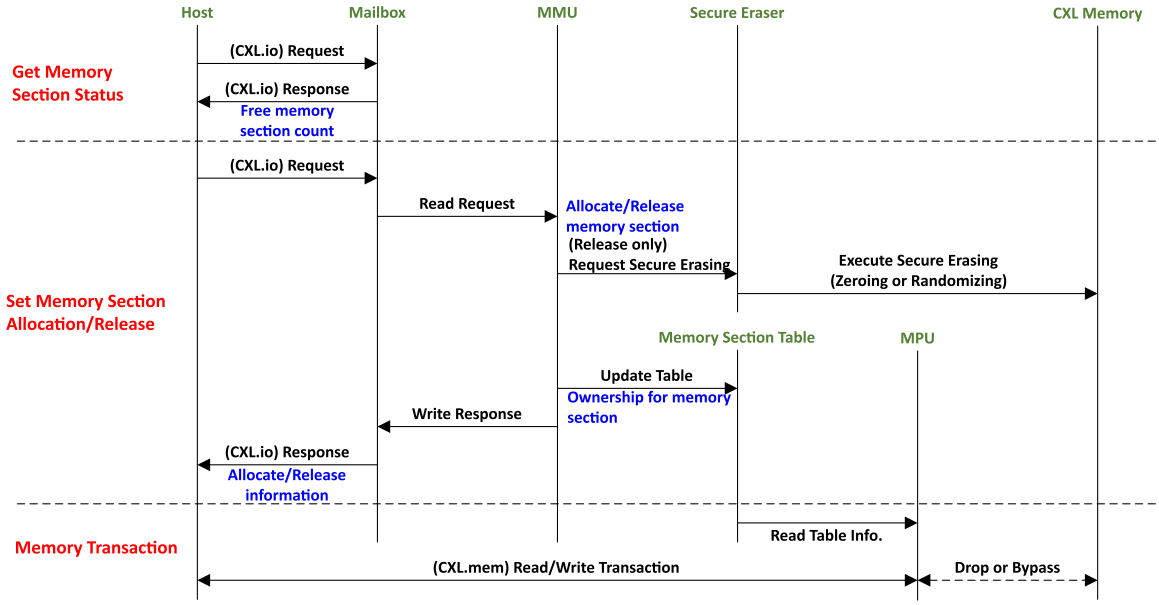

FIGURE 6. Communication flows of DCS APIs.

the DCS at Flash Memory Summit 2022.[11] Through a proof-of-concept platform based on a multiheaded field-programmable gate array (FPGA)-based platform, we demonstrate the potential benefits of DCS in CXL pooled memory system.

DCS Hardware Architecture

We designed a DCS engine to support dynamic capacity allocation and release, as shown in Figure 2. The DCS engine is composed of memory management unit (MMU), mailbox, memory section table, memory protection unit (MPU), and secure eraser.

Memory management unit: The CXL pooled memory is logically divided into a predefined size of memory section (128 MB by default), and MMU allocates or releases memory sections based on a host's request, as shown in Figure 3. ① MMU receives a memory allocation/release request from a host via mailbox. ② MMU allocates/releases requested memory sections by updating the memory section table. For a memory release request, secure–erase operation, such as zeroing or randomizing, is optionally performed to the requested memory sections. ③ MMU returns a response to the request, including execution status, the number of allocated or released sections, and the list of involved section identifiers (IDs).

Mailbox: Hosts and CXL pooled memory can communicate via mailbox compatible with CXL spec.[6] A host sends a 16-bit command described in Table 1 to the mailbox through the CXL.io path. We define custom DCS commands in the vendor-specific field allowed by CXL spec.[6] MMU reads the command and performs the corresponding operation. The detailed operation flows will be explained in the later section.

Memory section table: The structure of the memory section table, whose state is updated by MMU, is shown in Figure 4(a) describing the case of three hosts sharing the CXL pooled memory. The initial owner of the memory sections is set to 0, meaning no host can access the sections. Upon a host's memory allocation request, MMU allocates the requested number of memory sections and sets the sections' owner field to the host ID. On the other hand, MMU sets the requested sections' owner field to 0 upon a release request. When there are n hosts, the bit width of the owner field should be $\log_2(n+1)$.

Memory protection unit: MPU refers to the memory section table and decides whether to authorize a CXL. mem read/write transaction. As shown in Figure 4(b), MPU checks every memory access to the CXL pooled memory. The write request is dropped for unauthorized write and all 1 s are returned for unauthorized read.

Physical address space of CXL pooled memory is commonly exposed to all hosts. In the process of dynamically allocating a memory section using a DCS engine, an ownership of the section is stored in the memory section table. Hence, OS in each host can have an exclusive admission to each memory section. If the abovementioned operation is performed normally, there will be no host accessible without ownership. However, from a security point of view, a page

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

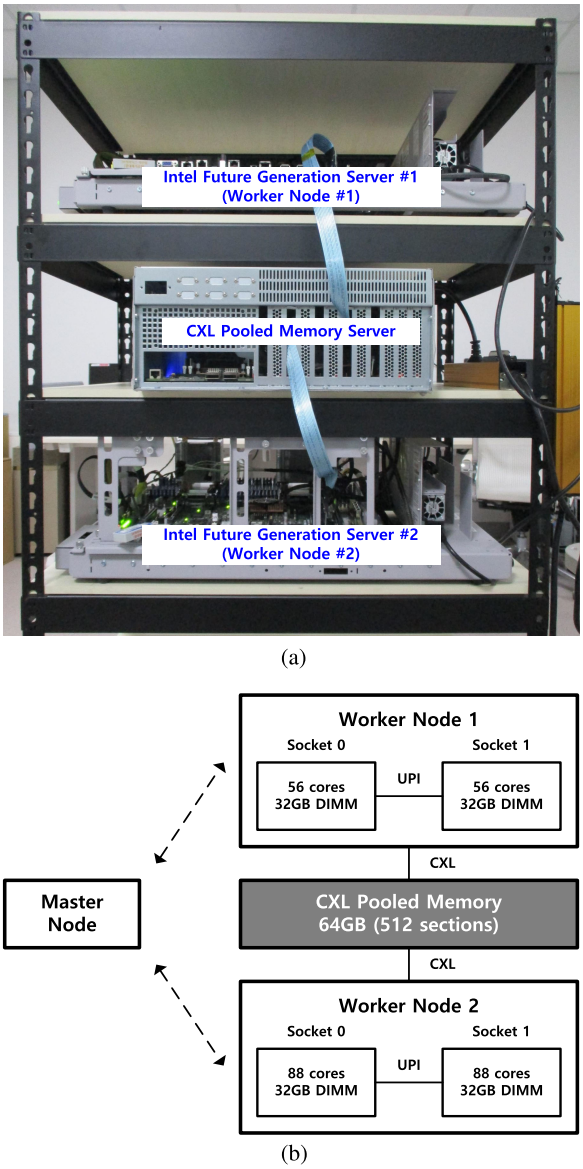

FIGURE 7. System configuration for the DCS evaluation. (a) FPGA-based CXL pooled memory server connected with Intel's future generation CPUs. (b) DCS-enabled Kubernetes cluster.

table can be set up to access unauthorized CXL pooled memory regions, which can be exploited maliciously, so device-level protection including ownership concept is essential. Although this can be implemented in hardware or software, in this study, it is implemented using MPU, a hardware module.

As far as we know, there is no information about memory protection in the recent CXL spec.[6] However, we think that it is necessary to discuss memory protection for the secure use of CXL pooled memory.

Secure eraser: Secure eraser performs zeroing or randomizing on the released memory sections before they are reallocated to other hosts. This feature prevents the content of the released memory sections from being exposed to malicious attacks.

DCS Software Framework

We designed DCS software framework composed of CXL pooled memory detection and management, DCS engine driver, and DCS-enabled Kubernetes.

CXL pooled memory detection and management: The physical address space for the CXL pooled memory is exposed as *soft-reserved*-type memory via EFI BIOS, and our customized device driver initialized the address space as a memory-only nonuniform memory access (NUMA) node (or zNUMA node). The unit of memory allocation or release in the CXL pooled memory is a *memory section*, the size of which is 128 MB by default on 64-bit x86 machines. DCS leverages Linux's memory hot-plug feature to register allocated CXL pooled memory as a system memory.

DCS engine driver: Upon the completion of the initialization phase, all memory sections are offline; thus, the memory space is unavailable to the Linux kernel. For a host to utilize CXL pooled memory, the host should get admission to access CXL pooled memory in the unit of memory section. Once the host receives admission, the admitted memory sections become online and are managed by the memory management subsystem in the Linux kernel. Communication between a host and the CXL pooled memory is enabled via CXL mailbox mechanism.

DCS-enabled Kubernetes: To manage pooled memory resource among multiple hosts, we adopt Kubernetes, a well-known container orchestration tool. As shown in Figure 5, Kubernetes cluster consists of Kubernetes master and multiple workers. To utilize the DCS feature of CXL pooled memory, we carefully modified Kubelet and device-plugin in Kubernetes worker. The device-plugin enumerates each sections of pooled memory as multiple memory devices to tracking the available memory resource, then shares the availability with Kubernetes master via Kubelet. The scheduler in Kubernetes master can select a worker to place a pod after checking the resource availability. If the selected worker does not have enough memory resource to run a container pod, the Kubernetes master sends allocation request to the worker. Once the worker receives the allocation request, the device-plugin do memory hot plug after obtaining admission of the section from CXL pooled memory. Upon completion of a pod execution, the device-plugin do unplug the sections and updates the resource availability.

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

DCS API

Host and CXL pooled memory device supporting DCS communicate using predefined application programming interface (API). This API supports three commands: `Get Memory Section Status`, `Set Memory Section Allocation`, and `Set Memory Section Release`. Figure 6 describes communication flows of DCS APIs.

Get memory section status: A host needs to know whether available memory sections exist in CXL pooled memory before requesting memory section allocation. `Get Memory Section Status` operates as follows. A host sends `Get Memory Section Status` command to mailbox. As a response to the command, mailbox returns the total memory section count and free section count with a success message.

Set Memory Section Allocation: Host requests CXL pooled memory to allocate memory sections considering the available memory sections within CXL pooled memory and the amount of memory capacity currently required or to release memory sections that are no longer needed. `Set Memory Section Allocation` operates in the following order.

1) Host requests `Set Memory Section Allocation` command with input payload, which are the required number of memory sections and allocation policy, to the mailbox.
2) MMU reads requested command.
3) Then MMU updates memory section table according to the input payload.
4) After updating, MMU returns response message, which are allocated section counts, extent counts, and section extent list.
5) The host reads the response message stored in the mailbox.

Set Memory Section Release: `Set Memory Section Release` operates similarly to `Set Memory Section Allocation`. The difference is secure erasing may be performed according to the release policy. If secure erasing is performed: 1) the MMU requests zeroing or randomizing to the secure eraser; and 2) the secure eraser performs zeroing or randomizing upon request. At this time, memory sections to which the secure eraser is applied cannot be reallocated.

FPGA Implementation

We implemented the DCS in an FPGA-based CXL pooled memory prototype connected to multiple hosts in Figure 2. The prototype can be connected to up to four hosts through the CXL interface, and is consisted of a DCS engine and four DDR4 DIMMs (up to 512 GB). As shown in Figure 7(a), the performance was evaluated by connecting two Intel's future generation CPU servers.

EVALUATION

We evaluate the memory utilization of two cases: 1) baseline case where the total pooled memory capacity is evenly allocated to each worker node; and 2) DCS-enabled case where the pooled memory is dynamically allocated and released to each worker node based on its request. Figure 7(b) describes the DCS-enabled Kubernetes cluster in our testbed consisting of a master node, two worker nodes, and a pooled memory prototype. Table 2 summarizes detailed system configuration of DCS-enabled Kubernetes.

To evaluate memory utilization, Kubernetes master node creates and schedules 11 pods, each of which is a single container running PageRank algorithm, and we used the built-in tool to generate random large graph as an input dataset. In our evaluation, each pod requires 24 CPU cores and 16 GB of memory. For the baseline, nine pods can initially run and two pods need to wait for the earlier pods to finish due to the memory shortage on the worker node 2. However, for the DCS-enabled system, all pods can run simultaneously without waiting.

As given in Table 2, compared with the baseline, DCS-enabled Kubernetes increases the cluster-wise

TABLE 2. System configuration and memory utilization of baseline and DCS-enabled Kubernetes.

	Baseline		**DCS-enabled Kubernetes**	
	Worker node 1	Worker node 2	Worker node 1	Worker node 2
# CPU cores	112	176	112	176
Local memory capacity	62 GB	62 GB	62 GB	62 GB
CXL memory capacity	32 GB	32 GB	64 GB (pooled)	
Total memory capacity	94 GB	94 GB	62–126GB	62–126GB
Average memory utilization	**55.7%**		**75.1%**	

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

memory utilization from 55.7% to 75% by providing memory capacity to memory-hungry hosts. The result shows that memory utilization is significantly improved when DCS is applied in the CXL pooled memory used by multiple hosts.

LESSONS LEARNED

Throughout the implementation of full software stack for DCS, we learned some lessons about how the current Linux kernel and cloud infra software, such as Kubernetes,[12] should evolve to better support CXL pooled memory systems.

Offlining of a memory section fails frequently: Before being released, memory section needs to be offlined but live pages within the victim section need to be migrated. The migration of live pages, however, often fails because the page is locked or being used at the moment. This incurs the overhead for selecting another victim section. To mitigate this issue, DCS employs the per-section counter for live pages, and selects a victim section with the smallest counter.

CXL pooled memory can be used as a slow-tier memory for multiple*fast-tier memory nodes:* Since it is of version 5.15, Linux kernel supports an option to demote cold pages in fast-tier (local) DRAM to ones in slow-tier (CXL) DRAM instead of swapping them out to SSDs. While a modern server equips with multiple memory NUMA nodes in the server, the current Linux design allows a slow-tier memory node to become the demotion node for only one fast-tier memory node. To address this issue, we have modified the Linux kernel to allow a slow-tier memory to become the demotion paths for multiple fast-tier memory nodes.

To enable QoS, a user needs a knob to control the capacity ratio between local DRAM and CXL pooled memory: QoS manageability under tiered memory systems is a prime concern to cloud service providers. In the current containerized environment, a user can only adjust the capacity limit for total memory usage for each memory controller group not for each of memory tiers. To address this issue, DCS provides a knob for limiting fast-tier memory usage, and thus, a user can control the memory capacity for each tiers.

Kubernetes manages system memory as a fixed resource: When the system memory size is changed by DCS, the memory.max value of cgroup, which controls the memory usage of pods (set of containers), is not updated. In situations where the system has enough free memory, the Kubelet may mistakenly terminate the pod. To resolve this issue, we adjusted the memory.max value according to section allocation and release. In addition, DCS can be used more easily in the cloud environment by automatically changing memory requests to section resources according to the situation at the scheduler stage of Kubernetes. However, Kubernetes prohibit changing the memory request size in the pod spec. To do this, we have modified the validation logic to allow the spec change in the API server of Kubernetes.

CONCLUSION

In this work, we have proposed DCS, an essential feature that CXL pooled memory should support, and implemented it for the first time in the industry. Through experiment using an FPGA-based CXL pooled memory prototype server, we have confirmed that DCS effectively maintains high utilization of CXL pooled memory.

THROUGH EXPERIMENT USING AN FPGA-BASED CXL POOLED MEMORY PROTOTYPE SERVER, WE HAVE CONFIRMED THAT DCS EFFECTIVELY MAINTAINS HIGH UTILIZATION OF CXL POOLED MEMORY.

Using our CXL pooled memory prototype server, pooled memory HW/SW solutions, such as block data management function and performance monitor above-mentioned, will be additionally applied, and studies focusing on improving performance of many servers will be conducted. Furthermore, we plan to design a CXL pooled memory appliance prototype with a CXL switch to support even larger memory capacity.

REFERENCES

1. A. N. Toosi, R. O. Sinnott, and R. Buyya, "Resource provisioning for data-intensive applications with deadline constraints on hybrid clouds using Aneka," *Future Gener. Comput. Syst.*, vol. 79, pp. 765–775, 2018.
2. C. Petersen and P. Chauhan, "Compute Express Link™(CXL™): Memory challenges and CXL solutions," CXL Consortium, 2020. Accessed: Feb. 1. [Online]. Available: https://www.computeexpresslink.org/post/join-us-compute-express-link-cxl-memory-challenges-and-cxl-solutions-webinar
3. K. Lim, J. Chang, T. Mudge, P. Ranganathan, S. K. Reinhardt, and T. F. Wenisch, "Disaggregated memory for expansion and sharing in blade servers," in *Proc. Int. Symp. Comput. Archit.*, 2009, pp. 267–278.

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.

4. P. Chauhan and M. Wagh, "CXL memory challenges," in *Proc. HotChips*, pp. 1–39, 2022.
5. H. Li et al., "Pond: CXL-based memory pooling systems for cloud platforms," in *Proc. Int. Conf. Archit. Support Program. Lang. Oper. Syst.*, 2023, pp. 1–17.
6. Compute Express Link Consortium, "Compute Express Link specification revision 3.0," 2022. Accessed: Feb. 1. [Online]. Available: https://www.computeexpresslink.org/download-the-specification
7. D. Gouk, S. Lee, M. Kwon, and M. Jung, "Direct access,{High-Performance} memory disaggregation with {DirectCXL}," in *Proc. USENIX Annu. Tech. Conf.*, 2022, pp. 287–294.
8. Z. Guo, Y. Shan, X. Luo, Y. Huang, and Y. Zhang, "Clio: A hardware-software co-designed disaggregated memory system," in *Proc. Int. Conf. Archit. Support Prog. Lang. Oper. Syst.*, 2022, pp. 417–433.
9. I. Calciu et al., "Rethinking software runtimes for disaggregated memory," in *Proc. Int. Conf. Archit. Support Prog. Lang. Oper. Syst.*, 2021, pp. 79–92.
10. I. Agarwal, "CXL overview and evolution," in *Proc. Hot Chips*, 2022, pp. 1–24.
11. J. Choi et al., "Elastic CXL memory solution: SK hynix advanced research project," *Flash Memory Summit*, 2022.
12. E. Truyen, N. Kratzke, D. Van Landuyt, B. Lagaisse, and W. Joosen, "Managing feature compatibility in Kubernetes: Vendor comparison and analysis," *IEEE Access*, vol. 8, pp. 228420–228439, 2020.

MINHO HA is a staff engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include memory-centric computing architecture and hardware efficient deep learning architecture. Contact him at minho1.ha@sk.com.

JUNHEE RYU is a principal engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include operating systems, real-time systems, and storage systems. Contact him at junhee.ryu@sk.com.

JUNGMIN CHOI is a senior staff engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include CXL pooled memory architecture and near memory processing. Contact him at jungmin.choi@sk.com.

KWANGJIN KO is a staff engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include operating systems and kernel memory management. Contact him at kwangjin.ko@sk.com.

SUNWOONG KIM is a senior staff engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include the server architecture and CXL interconnect. Contact him at sunwoong1.kim@sk.com.

SUNGWOO HYUN is a senior staff engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include memory disaggregation, DRAM row hammer, and prefetch. Contact him at sungwoo.hyun@sk.com.

DONGUK MOON is a senior staff engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include processing-in-memory, processing-near-memory, and pooled memory solutions. Contact him at donguk.moon@sk.com.

BYUNGIL KOH is a principal engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include CXL memory architecture and memory-centric computing architecture. Contact him at byungil.koh@sk.com.

HOKYOON LEE is a senior staff engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include system software and cloud computing for CXL memory. Contact him at hokyoon.lee@sk.com.

MYOUNGSEO KIM is a principal engineer at SK hynix Inc., Icheon, 17336, South Korea. His research interests include CXL memory solution, computer architecture, and memory systems. Contact him at myoungseo.kim@sk.com.

HOSHIK KIM is a VP at SK hynix Inc., Icheon, 17336, South Korea. His research interests include next-generation memory system architecture and software solution for data centers at scale, such as CXL memory expansion, memory disaggregation, and near-memory processing. Contact him at hoshik.kim@sk.com.

KYOUNG PARK is an SVP at SK hynix Inc., Icheon, 17336, South Korea. His research interests include cloud computing, memory and storage architecture, and in-memory (and storage) processing. Contact him at kyoung.park@sk.com.

Authorized licensed use limited to: SKHynix (f: Hynix). Downloaded on March 09,2023 at 13:24:19 UTC from IEEE Xplore. Restrictions apply.